COMPUTER ENGINEERING

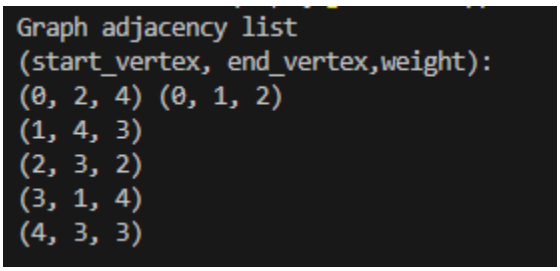
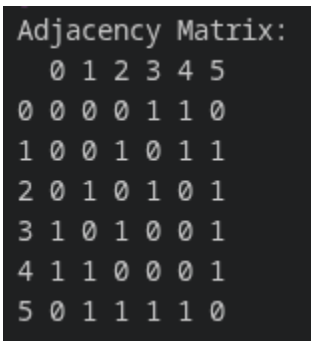
QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	10/2/2026	**DATE SUBMITTED:**	10/2/26
NAME:	Condino, Niel Vincent B.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 10.1 GRAPHS

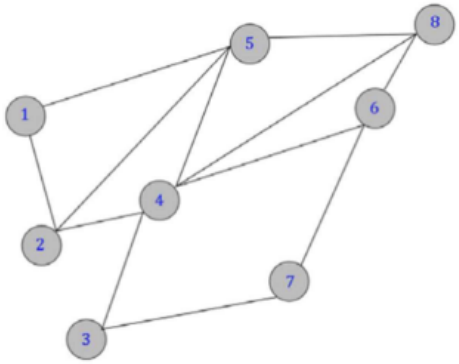
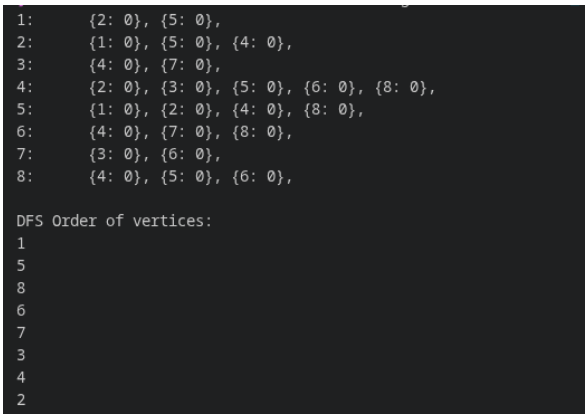
1. PROCEDURE OUTPUT

Table 1: Graph Adjacency List and Graph Adjacency Matrix

Adjacency List	
Adjacency Matrix	

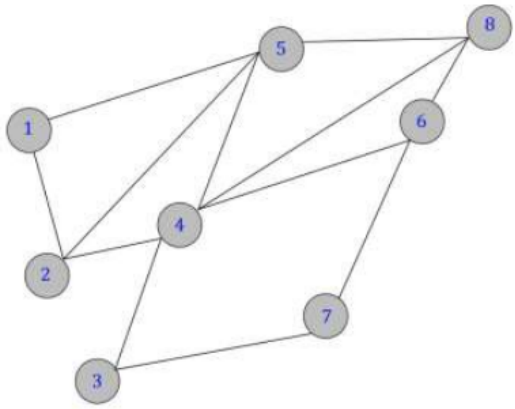
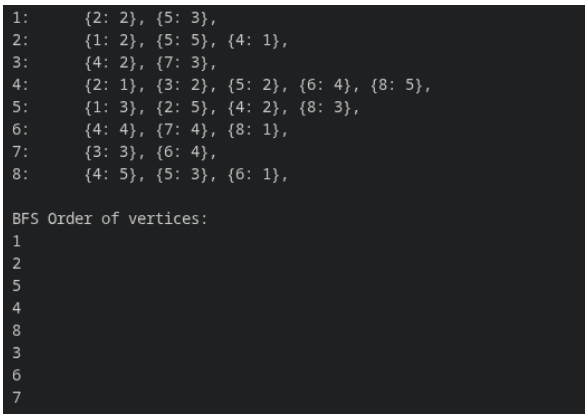
The table above shows the implementation of a graph adjacency list and adjacency matrix. Both of these are data structures used to represent a graph internally. An adjacency list uses a list to keep track of the vertex and all of its neighboring vertices, along with the weight of the edges leading to them. An adjacency matrix uses a 2x2 array or a matrix to represent a graph. The value in the intersection of the row and column indices is used to represent whether an edge or connection exists between vertices. 1 if it exists and 0 if not. These values can be changed when representing a weighted graph, using the weight as values in these indices.

Table 2:Depth-First Search Implementation C++

Graph	Code Output
	

The output above shows the testing and usage of depth-first search on the graph shown in the image. I observed that it first shows the graph's adjacency list. This list shows that the graph is unweighted and undirected. Below this, the order in which the vertices are visited is shown, still following the depth-first strategy. Starting from vertex 1, it follows this traversal: 1->5->8->6->7->3->4->2. Because they are internally put inside a stack, the program goes through each vertex in the stack, keeping track of the visited nodes so that they are not visited again.

Table 3: Breadth-First Search Implementation C++

Graph	Output
	

The output above shows the testing and usage of breadth-first search on the graph shown in the image. The first lines show the internal representation of the graph. This reveals that the graph is an undirected and weighted graph. Below this, the order in which the vertices are visited is shown, still following the breadth-first strategy.

Starting from vertex 1, it follows this traversal: 1->2->5->4->8->3->6->7. It traverses the graph level by level. Before it proceeds down the graph, it first checks all the neighboring vertices. It also traverses by prioritizing the edges that weigh less.

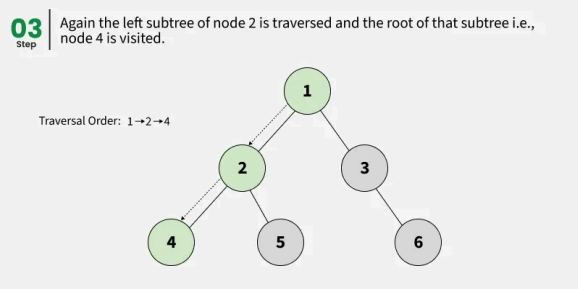
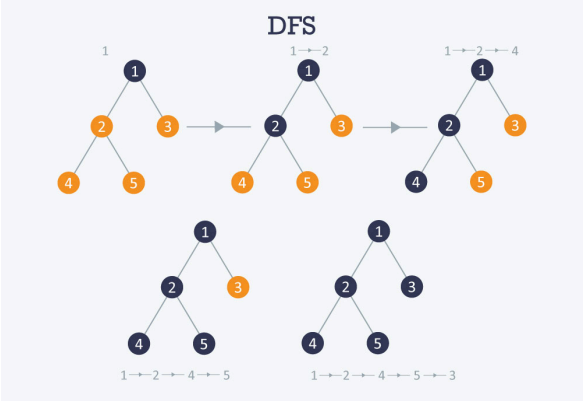
2. ANSWERS TO SUPPLEMENTAL ACTIVITY

1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until he finishes from one vertex, backtracks, and then explores other vertices from the same vertex. Discuss which algorithm would be most helpful to accomplish this task.

The algorithm that will help him achieve his desired goal is a depth-first search. In this algorithm, the person prioritizes depth: going down until they reach the end of that path, then backtracking to the last split point where other paths are available. This way, this person can visit all the other locations.

2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.

The equivalent of DFS traversal in trees is the pre-order traversal. Like pre-order traversal, the algorithm visits a node, then repeatedly goes "left" until it reaches a leaf, returns that node, and backtracks. DFS is similar to this as it also finds or visits every vertex/node until it reaches the end of the graph(prioritizing depth), then backtracks to check other paths.

Pre-Order Traversal	Depth First Search
 Reference:https://media.geeksforgeeks.org/wp-content/uploads/20251001114609429721/preorder_traversal_of_binary_tree_9.webp	 Reference:https://he-s3.s3.amazonaws.com/media/uploads/9fa1119.jpg

3. In the performed code, what data structure is used to implement the Breadth First Search?

In the performed code, the data structure used to implement it is a queue. This is used because breadth is prioritized and the graph is traversed level by level. This ensures the first adjacent vertex discovered is the first one processed. The queue holds back these lower-level vertices in line and ensures they are processed after the level above it is discovered.

4. How many times can a node be visited in the BFS?

In both algorithms, each node is visited only once. This is because in each algorithm, we keep track of the visited vertices. These visited nodes are prevented from being enqueued again so that we prevent unnecessary processing and longer infinite loops in case the graph is cyclic.

Conclusion:

In this activity, I was able to learn about graphs. Graphs have nodes also known as vertices, that are connected by edges. This can be internally represented in 2 ways: an adjacency list or an adjacency matrix. An adjacency list stores the start vertex, end vertex, and weight of each edge. An adjacency matrix uses indices of a 2x2 matrix to store the weight of the edges connecting different vertices. This data structure is non-linear, meaning there is no definitive sequential order. To traverse, we used 2 algorithms: Depth-First Search and Breadth-First Search. Depth-first prioritizes depth, so this traversal tries to find the end of the graph before backtracking to find other paths. Breadth-First prioritizes breadth, meaning the graph is traversed level by level. Overall, I learned many things about graphs, and I want to see this implemented as a solution to different problems in the future.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.